

Project 8: Interpolation, extrapolation, and climate change

Goals: To explore applications of linear algebra for the interpolation of data and prediction of future tendencies.

To get started:

- Create a new Python script and save it as lab08.py
- Download the file temperature.mat which contains average temperatures for the past 136 years ⁵. Temperature is given in Celsius.

Python commands used: matplotlib.pyplot.plot, matplotlib.pyplot.axis, numpy.vander, numpy.interp, numpy.linalg.orth

What you have to submit: The file lab08.py, which you will create during the lab session.

INTRODUCTION

In many practical applications it is necessary to obtain an intermediate value of a parameter from a few known data points. If this intermediate value lies in between two known measurements, we will call this process interpolation. If the value is outside of the sampled interval, then we will call this process extrapolation. In this project we will look at several procedures for interpolation and extrapolation and apply them to weather and climate change prediction.

The most common reason for using interpolation is not having enough of data points. These data points can be obtained from experimental observations which can be expensive or difficult to perform, or from past observations where the data is simply unavailable. Interpolation is also used when the function which produces the data is known but has a complicated structure. In this case it can be computationally prohibitive to use this function, especially if, in the course of computation, this function needs to be called multiple times. For instance, the function can be given by a slowly convergent series which requires computing many terms to obtain adequate accuracy. Approximating this complicated function by a simpler function may be helpful. Interpolation of a function can be sufficiently accurate provided that the function is smooth and there are enough data points available. Some of the usual types of interpolation include polynomial interpolation when the function is approximated by a polynomial, trigonometric approximation with a truncated Fourier series, or various piece-wise approximations. More intricate interpolation methods can be used if additional information is available about the approximated function.

Extrapolation, on the other hand, is a significantly more difficult procedure. The general rule is not to extrapolate too far beyond the range of the observed parameters. There are multiple reasons for this. For instance, some additional parameters may affect the behavior of a function outside of the observed range of values. Imagine trying to extrapolate physical laws applicable for regular macro-solids (such as things we use in our daily life) to nano-scale or to astro-scales. The obtained results will be meaningless since different laws apply at those scales. Greater

⁵ The temperature data has been taken from NASA's website, <https://climate.nasa.gov/vital-signs/global-temperature/>

uncertainty and errors while extrapolating also appear because small errors in data measurements can potentially lead to very large errors while extrapolating the data. These errors are somewhat bounded while interpolating between two known data points.

TASKS

1. To begin, let us look at interpolation based on the polynomial approximation. Assume that we have $n+1$ data points (measurements) (x_i, y_i) , $i = 0, \dots, n$. To obtain the values of the function in between these data points we can simply trace a polynomial of the n th degree through the data points and use this polynomial to estimate the values of a function in between measurements. Thus, the goal is to find a polynomial

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$$

with unknown coefficients a_i , $i = 0, \dots, n$, which passes through the points (x_i, y_i) . Then the coefficients a_i must satisfy the following system of linear algebraic equations

$$a_n x_i^n + a_{n-1} x_i^{n-1} + \dots + a_1 x_i + a_0 = y_i, \quad i = 0, \dots, n. \text{ We can write}$$

this system in the form

$$\mathbf{V}\mathbf{z} = \mathbf{y},$$

where V is the so-called Vandermonde matrix:

$$\mathbf{V} = \begin{bmatrix} 1 & x_0 & x_0^2 & \dots & x_0^{n-1} & x_0^n \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 1 & x_1 & x_1^2 & \dots & x_1^{n-1} & x_1^n \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^{n-1} & x_n^n \end{bmatrix},$$

$\mathbf{z} = [a_0, a_1, \dots, a_n]^T$ is the unknown vector of polynomial coefficients and $\mathbf{y} = [y_0, y_1, \dots, y_n]^T$ is the known vector of the function values at sample points.

We will start by approximating the data for the average temperatures in Kansas⁶:

Month	1	2	3	4	5	6	7	8	9	10	11	12
Temperature (High)	37	44	55	66	75	84	89	88	80	69	53	41
Temperature (Low)	17	23	33	43	53	63	68	65	56	44	32	22

In our computations, we will work with high temperatures only. To begin, let us generate a plot of the temperatures in Python.

Observe that Python will automatically join the available data points by straight lines.

Variables: WeatherHigh

⁶ Source for the temperature data: Wikipedia <https://en.wikipedia.org/wiki/Kansas#Climate>, temperature is given in Fahrenheit.

```
# High temperatures in Kansas (in Fahrenheit)
WeatherHigh = np.array([37, 44, 55, 66, 75, 84, 89, 88, 80, 69, 53, 41])
```

```
# Plot the temperatures
plt.figure()
plt.plot(range(1, 13), WeatherHigh, 'r-x')
plt.axis([1, 12, 30, 95])
plt.title('Average High Annual Temperatures in Kansas')
plt.xlabel('Month')
plt.ylabel('Temperature (F)')
plt.grid(True)
plt.show()
```

2. Assume for now, that we are only given the temperatures for four months: January, May, August, and December. Select the corresponding data from the variable WeatherHigh above and generate the Vandermonde matrix based on this data. Now solve the resulting system using Python's backslash linear system solver:
The resulting vector CoefHigh contains coefficients of the cubic polynomial approximating the average high annual temperature in Kansas. **Variables: V, CoefHigh**

```
# Months: January, May, August, December
x = np.array([1, 5, 8, 12])
V = np.vander(x, increasing=True)
```

```
# Select corresponding temperatures
y = WeatherHigh[[0, 4, 7, 11]]
```

```
# Solve for polynomial coefficients
CoefHigh = lstsq(V, y, rcond=None)[0]
# Given months: January, May, August, and December
x = np.array([1, 5, 8, 12])
y = WeatherHigh[x-1] # Select corresponding temperatures
```

```
# Generate the Vandermonde matrix
V = np.vander(x, increasing=True)
```

```
# Solve for polynomial coefficients
CoefHigh = np.linalg.solve(V, y)
```

```
print("Vandermonde Matrix (V):")
print(V)
print("Coefficients of the cubic polynomial (CoefHigh):")
print(CoefHigh)
```

```
print("Coefficients of the cubic polynomial:", CoefHigh)
```

3. Python's `numpy.polyval` function can be used to efficiently evaluate a polynomial at the given set of points. It takes two vectors as arguments. The first vector represents the coefficients of the polynomial and the second vector represents the points we want to evaluate the polynomial at. Remember that you can always type `help(np.polyval)` or `np.polyval?` in the Python command window to see more options. Let us plot the graph of the polynomial obtained at the previous step by using the following code:

This graph shows the difference between the approximated values and the exact values. Observe that the command `axis([1,12 30 95])` sets the parameters for x and y axes. Namely, the x axis will range from $x = 1$ to $x = 12$ and the y axis will range from $y = 30$ to $y = 95$.

Variables: `xc, ycHigh`

```
# Evaluate the polynomial at the given set of points
xc = np.arange(1, 12.1, 0.1)
ycHigh = np.polyval(CoefHigh[::-1], xc) # CoefHigh needs to be reversed for np.polyval

# Plot the polynomial and the original data
plt.figure()
plt.plot(xc, ycHigh, 'b-', label='Cubic Polynomial')
plt.plot(range(1, 13), WeatherHigh, 'r-x', label='Exact Data')
plt.axis([1, 12, 30, 95])
plt.xlabel('Month')
plt.ylabel('Temperature (High)')
plt.title('Polynomial Approximation of High Temperatures')
plt.legend()
plt.grid(True)
plt.show()
```

4. Repeat the steps above to produce a polynomial approximation based on the temperatures from the following six months: January, March, May, August, October, and December. You will need to use a polynomial of the fifth degree. Plot the resulting polynomial next to the exact values as in the step above. You will see that this approximation produces much closer results to the exact values.

```
# Given months: January, March, May, August, October, and December
x_six = np.array([1, 3, 5, 8, 10, 12])
y_six = WeatherHigh[x_six - 1] # Select corresponding temperatures

# Generate the Vandermonde matrix
V_six = np.vander(x_six, increasing=True)
```

```

# Solve for polynomial coefficients
CoefHigh_six = np.linalg.solve(V_six, y_six)

print("Vandermonde Matrix for six months (V_six):")
print(V_six)
print("Coefficients of the 5th degree polynomial (CoefHigh_six):")
print(CoefHigh_six)

# Evaluate the polynomial at the given set of points
ycHigh_six = np.polyval(CoefHigh_six[::-1], xc) # CoefHigh_six needs to be reversed for
np.polyval

# Plot the polynomial and the original data
plt.figure()
plt.plot(xc, ycHigh_six, 'b-', label='5th Degree Polynomial')
plt.plot(range(1, 13), WeatherHigh, 'r-x', label='Exact Data')
plt.axis([1, 12, 30, 95])
plt.xlabel('Month')
plt.ylabel('Temperature (High)')
plt.title('5th Degree Polynomial Approximation of High Temperatures')
plt.legend()
plt.grid(True)
plt.show()

```

5. Finally, let us use the temperatures from all twelve months. You will need to use a polynomial of the eleventh degree this time. Modify the code above and run it in Python. If you did everything correctly, you will obtain a picture which looks similar to the one in Fig. 7. You may notice that this approximation is not necessarily better than the

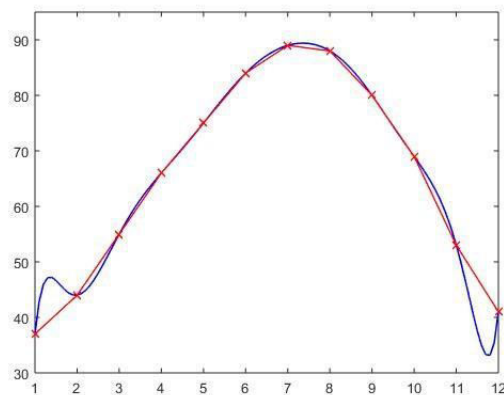


Figure 7: Approximation with the 11th degree polynomial

approximation with the 5th degree polynomial due to large fluctuations near the end points. This is typical for approximations with polynomials of a high degree on evenly spaced grids which tend to oscillate widely near the end points. This is one of the reasons why polynomials of a high degree are normally not used for interpolation on evenly spaced grids⁷. Additionally, if you look at the Python output in the command window, you will probably see something like this

```
Warning: Matrix is close to singular or badly scaled. Results may be inaccurate. RCOND = 8.296438e-17.
```

This is due to the fact that Vandermonde matrix V is an ill-conditioned matrix, meaning that the system of linear equations with this matrix is difficult to solve with high accuracy. The matrix V becomes more ill-conditioned as the dimensions of the matrix increase. This is another reason why high degree polynomial approximations on evenly spaced grids are rarely used in interpolation.

```
# All twelve months
x_all = np.arange(1, 13)
y_all = WeatherHigh

# Generate the Vandermonde matrix
V_all = np.vander(x_all, increasing=True)

# Solve for polynomial coefficients
CoefHigh_all = np.linalg.solve(V_all, y_all)

print("Vandermonde Matrix for all twelve months (V_all):")
print(V_all)
print("Coefficients of the 11th degree polynomial (CoefHigh_all):")
print(CoefHigh_all)

# Evaluate the polynomial at the given set of points
ycHigh_all = np.polyval(CoefHigh_all[::-1], xc) # CoefHigh_all needs to be reversed for
np.polyval

# Plot the polynomial and the original data
plt.figure()
plt.plot(xc, ycHigh_all, 'b-', label='11th Degree Polynomial')
plt.plot(range(1, 13), WeatherHigh, 'r-x', label='Exact Data')
plt.axis([1, 12, 30, 95])
plt.xlabel('Month')
plt.ylabel('Temperature (High)')
```

⁷ However, approximations with the polynomials of high degree on non-evenly spaced grids may be extremely useful in computations and achieve very high precision. Chebyshev nodes and Chebyshev polynomials represent a good example of this phenomena.

```
plt.title('11th Degree Polynomial Approximation of High Temperatures')
plt.legend()
plt.grid(True)
plt.show()
```

6. If polynomials of a high degree on evenly spaced grids are not good for interpolation, what can be done instead? The most common method of interpolation is through the approximation of functions with piece-wise (typically polynomial) functions. This means that the function between data points is approximated by a small degree polynomial (polynomials of no higher than third degree are usually used). These polynomials are different on each of the intervals between the data points. Individual approximations are then glued together to achieve a desired degree of smoothness at the data points. Under smoothness here we understand the continuity of a function and its derivatives up to the some order. Smoothness of the resulting approximation depends on the degree of polynomials used and how the polynomial pieces are glued together. If a higher degree of smoothness is desired, it is necessary to use polynomials of a higher degree. For instance, if we use cubic polynomials, then it is possible to achieve continuity of the function itself and its first two derivatives at the most.

To investigate this method of interpolation, we will look at the built in Python function `interp1`. This function allows one to interpolate a one-dimensional function. It has several attributes which allow for different types of interpolation: “linear” which simply joins the data points with straight lines, “cubic” or “pchip” which approximates the function on each individual segment with the third degree polynomial, and “spline” which also uses third degree polynomials, but with different junction conditions. So what is the difference between “pchip” and “spline”? Simply put, the “pchip” method provides a shape-preserving piece-wise cubic polynomial function which is smooth together with its first derivative. The “spline” interpolation creates an “extra-smooth” piece-wise cubic polynomial function which is smooth together with its first and second derivatives. In general, “pchip” approximations have smaller oscillations near the ends and are easier to compute.

Compare these options by using the following code:

Here the variables `x` and `WeatherHigh` give the vectors of initial data. The vector `xc` is a vector of points where we want to approximate the function and the vectors `ycHigh1`, `ycHigh2`, and `ycHigh3` are the values at the points `xc` resulting from the interpolation. Plot the vectors `ycHigh1`, `ycHigh2`, and `ycHigh3` versus `xc` on the same plot to observe the smoothness of the resulting data. You can see that `ycHigh2` and `ycHigh3` achieve better approximations than the polynomial approximation. The function `interp1` also has a possibility to extrapolate the results which we will explore shortly. Variables: `ycHigh1`, `ycHigh2`, `ycHigh3`

```
import numpy as np
import matplotlib.pyplot as plt

# Given data
x = np.arange(1, 13)
```

```

WeatherHigh = np.array([53, 60, 68, 77, 85, 90, 92, 89, 82, 71, 60, 55])

# Points for interpolation
xc = np.arange(1, 12.1, 0.1)

# Linear interpolation
ycHigh1 = np.interp(xc, x, WeatherHigh, left=None, right=None, period=None)

# Piecewise cubic Hermite interpolating polynomial (PCHIP)
from scipy.interpolate import PchipInterpolator
pchip_interpolator = PchipInterpolator(x, WeatherHigh)
ycHigh2 = pchip_interpolator(xc)

# Cubic spline interpolation
from scipy.interpolate import CubicSpline
spline_interpolator = CubicSpline(x, WeatherHigh)
ycHigh3 = spline_interpolator(xc)

# Plot the interpolations
plt.figure()
plt.plot(xc, ycHigh1, 'g-', label='Linear Interpolation')
plt.plot(xc, ycHigh2, 'r-', label='PCHIP Interpolation')
plt.plot(xc, ycHigh3, 'k-', label='Spline Interpolation')
plt.plot(x, WeatherHigh, 'bo', label='Original Data')
plt.axis([1, 12, 30, 95])
plt.xlabel('Month')
plt.ylabel('Temperature (High)')
plt.title('Comparison of Interpolation Methods')
plt.legend()
plt.grid(True)
plt.show()

```

7. Now let us look at the extrapolation of the data. Extrapolation means finding an approximation to the data outside of the initial measurement interval. For our data, we will take the average global temperatures from the last 136 years starting from the year 1880⁸. To begin, load the matrix temperature from the file “temperature.mat” using the load command.

Variables: temperature

```

import scipy.io
from scipy.io import loadmat

```

⁸ The source of data: NASA, <https://climate.nasa.gov/vital-signs/global-temperature/>

```
# Load temperature data
data = loadmat('C:/Users/Harshith/Downloads/laproject/materials/temperature.mat')
temperature = data['temperature']
```

8. The matrix `temperature` consists of two columns. The first column represents the years and the second the average temperatures at those years. Separate the data into two variables:

Plot the graph of temperatures using the `plot` function. This will produce a graph of fluctuations of average temperatures since 1880. If you did everything correctly, you should obtain a graph similar to the one on the Fig. 8. **Variables: years, temp**

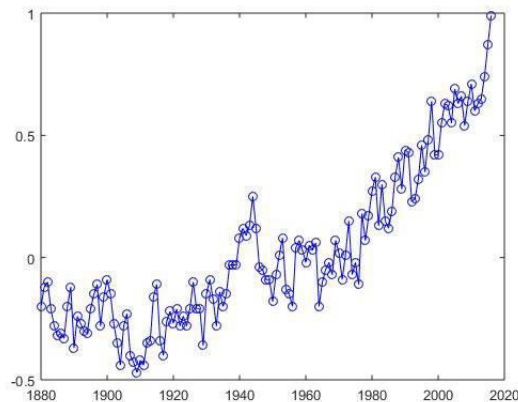


Figure 8: Average yearly temperatures.

```
# Separate data into years and temperatures
```

```
years = temperature[:, 0]
```

```
temp = temperature[:, 1]
```

```
# Plot the temperature data
```

```
plt.figure()
```

```
plt.plot(years, temp, 'b-o')
```

```
plt.xlabel('Year')
```

```
plt.ylabel('Temperature (Celsius)')
```

```
plt.title('Average Yearly Temperatures')
```

```
plt.grid(True)
```

```
plt.show()
```

9. Let us attempt to extrapolate the data in the future. Assume that we want to know the temperature about 10 years into the future (again, it is advised not to attempt to extrapolate too far beyond the initial interval). We can do this using the following code:

This produces three graphs based on the different interpolation/extrapolation strategies which were discussed above. If you did everything correctly, you should obtain three radically different predictions for the next 10 years!

Variables: futureyears, futuretemp1, futuretemp2, futuretemp3

```
# Define the future years
futureyears = np.arange(2016, 2026)

# Linear extrapolation
linear_interpolator = interp1d(years, temp, kind='linear', fill_value='extrapolate')
futuretemp1 = linear_interpolator(futureyears)

# Piecewise cubic Hermite interpolating polynomial (PCHIP)
pchip_interpolator = PchipInterpolator(years, temp, extrapolate=True)
futuretemp2 = pchip_interpolator(futureyears)

# Cubic spline interpolation
spline_interpolator = CubicSpline(years, temp, extrapolate=True)
futuretemp3 = spline_interpolator(futureyears)

# Plot the extrapolated data
plt.figure()
plt.plot(years, temp, 'b-o', label='Original Data')
plt.plot(futureyears, futuretemp1, 'g-o', label='Linear Extrapolation')
plt.plot(futureyears, futuretemp2, 'r-x', label='PCHIP Extrapolation')
plt.plot(futureyears, futuretemp3, 'k-d', label='Spline Extrapolation')
plt.xlabel('Year')
plt.ylabel('Average Temperature (°C)')
plt.title('Extrapolation of Average Yearly Temperatures')
plt.legend()
plt.grid(True)
plt.show()
```

10. So what happened in the previous example? The local fluctuations in temperature significantly affected the global predictions for the future. However, we are not interested in the local fluctuations but rather in global tendencies. So let us try to separate those two from each other and extrapolate based on global tendencies only. We can think of temperatures in the following terms: $t_i = t(\text{year}_i) + \text{noise}$,

where $t(\text{year})$ would be the global tendency of temperature and *noise* represents relatively small year-to-year fluctuations.

We will use orthogonal projections to separate the global behavior from the “noise”. As a first approximation, let us consider how we can assess the overall average temperature. That is, we approximate the temperature $t(\text{years})$ as a constant:

$$t(\text{years}) \approx \frac{1}{n} \sum_{i=1}^n t_i$$

Enter the following command in the command window

```
>> sum(temp)/n
```

This allows us to see that the average temperature for the past 136 years was 0.0244C.

11. Here is another way of thinking of the same operation using orthogonal projections. If the data was truly flat, then there would be a real number c so that

$$\begin{bmatrix} t_1 \\ t_2 \\ \vdots \\ t_n \end{bmatrix} = c \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}$$

$$\mathbf{t} = \begin{bmatrix} t_1 \\ t_2 \\ \vdots \\ t_n \end{bmatrix} = c \mathbf{1} = c \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}$$

That is, the average temperature vector $\mathbf{t}$ would be a scalar times the vector $\mathbf{1}$ of all ones. Since the data is not flat, we can't find such a c . However, we can do the next best thing. We can find c to minimize the norm $\|\mathbf{t} - c\mathbf{1}\|$. The best choice for c is determined by the orthogonal projection on a vector $\mathbf{1}$:

$$c = \frac{\mathbf{1}^T \mathbf{t}}{\mathbf{1}^T \mathbf{1}} \quad \text{and for this choice of } c, \quad c\mathbf{1} = \frac{\mathbf{1}^T \mathbf{t}}{\mathbf{1}^T \mathbf{1}} \mathbf{1} = \text{proj}_{\mathbf{1}}(\mathbf{t}).$$

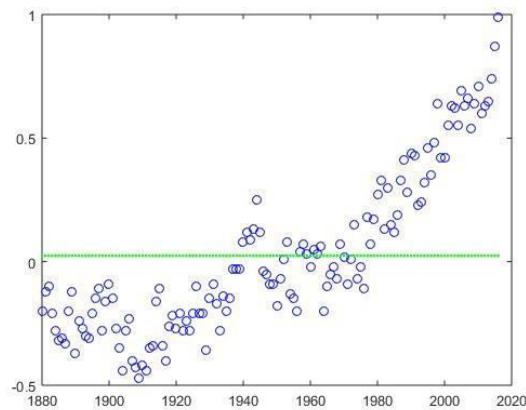


Figure 9: Approximation of data by a constant function.

Moreover, a little rearranging shows that

$$\text{proj}_{\mathbf{1}}(\mathbf{t}) = \left(\frac{1}{\|\mathbf{1}\|^2} \mathbf{1}\mathbf{1}^T \right) \mathbf{t} = \mathbf{P}\mathbf{t} \quad \text{where} \quad \mathbf{P} = \frac{1}{\|\mathbf{1}\|^2} \mathbf{1}\mathbf{1}^T \quad \text{is the projection matrix.}$$

Define the variable `b1` to be the `length(temp)`-dimensional **1** vector. In the space after that, define the variable `P1` to hold the projection matrix. You will either want to use $\sqrt{\mathbf{u}^T \mathbf{u}}$

Python's `norm` function, or remember that $\mathbf{u} \mathbf{u}^T = \mathbf{u} \mathbf{u}^T$. Also, define the variable `temp1` to be the value `P1*temp`. **Variables: `P1`, `temp1`**

12. Plot the resulting vector using the code:

You should see a similar plot as before, but now with a set of green dots representing the projection (Fig. 9).

13. Now, examine the value of `temp1` in the Python command window.

Q1: How does the value of `temp1` relate to the average value of temperature?

Solution for 10,11,12,13 is combined.

```
# Calculate the average temperature
average_temp = np.mean(temp)
```

```
# Print the average temperature
print(f"The average temperature for the past 136 years was {average_temp:.4f}°C.")
```

```
# Calculate orthogonal projection
n = len(temp)
b1 = np.ones(n)
P1 = np.outer(b1, b1) / np.dot(b1, b1)
temp1 = P1 @ temp
```

```
# Print projection matrix and projected temperatures
print("Projection matrix P1:")
print(P1)
print("Projected temperatures (temp1):")
print(temp1)
```

```
# Plot the results
plt.figure()
plt.plot(years, temp, 'bo', label='Original Data')
plt.plot(years, temp1, 'g.', label='Projected Data')
plt.xlabel('Year')
plt.ylabel('Temperature Anomaly (°C)')
plt.title('Temperature Data Projection')
plt.legend()
plt.grid(True)
plt.show()
```

14. Like vectors, matrices also have norms. The norm $\|A\|$ of a matrix A describes how far A is from the zero matrix in a certain sense. Enter the following command into Python

Your answer should be something like `ans = 2.0134e-15`.

Q2: What does the result say about the relationship between the projection matrix P and its square P^2 ?

```
# Norm of P^2 - P
norm_P1 = np.linalg.norm(P1 @ P1 - P1)
print(f"norm(P1 * P1 - P1) = {norm_P1:.4e}")
```

15. While the previous approach of approximating the data with a constant function may be acceptable in some cases, it is clearly not a perfect match. Let us try instead to model the data as a linear function of *years*:

$$t_i = b \text{years}_i + c + \text{noise}.$$

If this formula was exact (with no noise), we would have that

$$\mathbf{t} = \begin{bmatrix} t_1 \\ t_2 \\ \vdots \\ t_n \end{bmatrix} = \begin{bmatrix} \text{years}_1 & 1 \\ \text{years}_2 & 1 \\ \vdots & \vdots \\ \text{years}_n & 1 \end{bmatrix} \begin{bmatrix} b \\ c \end{bmatrix} = \mathbf{years} + c\mathbf{1},$$

so $\mathbf{t}$ would be a vector in the subspace $\text{span}\{\mathbf{years}, \mathbf{1}\}$. However, as before, this is not exactly true. So, we will need to use an orthogonal projection again.

There is a nice formula for projecting onto a subspace S generated by a span of several vectors $\text{span}\{\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_k\}$, but it requires that the vectors be *orthonormal*. That is $\|\mathbf{b}_i\| = 1$ for each i , and $\mathbf{b}_i^T \mathbf{b}_j = 0$ for all $i \neq j$.

One way to produce an orthonormal basis in Python is to use the Python `orth` function. Please, type in the code

This creates a matrix $B = [\mathbf{1} \ \mathbf{years}]$, whose first column consists entirely of ones and whose second column is the vector $\mathbf{years}$, which provides the years of observation. The new array `Q2` is defined to be the result of `orth(B2)`.

The function `orth` will create an orthonormal basis (defined by the columns of `Q2`) for the subspace S . To check that the columns of `Q2` form a basis, verify in the command window that the values of `rank(Q2)` and `rank([Q2, B2])` are both equal to 2.

Variables: `Q2`

Q3: What kind of matrix is the matrix $Q2^T Q2$? Why?

```
from scipy.linalg import orth
# Create the matrix B2
```

```

m = len(years)
B2 = np.column_stack((np.ones(m), years))

# Create the orthonormal basis using orth
Q2 = orth(B2)

# Verify the ranks
rank_Q2 = np.linalg.matrix_rank(Q2)
rank_Q2_B2 = np.linalg.matrix_rank(np.column_stack((Q2, B2)))

print(f"Rank of Q2: {rank_Q2}")
print(f"Rank of [Q2, B2]: {rank_Q2_B2}")

# Q3: What kind of matrix is Q2^T Q2? Why?
Q2_T_Q2 = np.dot(Q2.T, Q2)
print("Q2.T @ Q2:")
print(Q2_T_Q2)

```

16. Given an orthonormal basis defined by the columns of $Q2$ for the subspace S , the formula for the projection matrix onto the subspace S is simple:

$$\mathbf{P2} = \mathbf{Q2Q2}^T.$$

Create a new cell and in this cell create a variable $P2$ to hold the value of the matrix of the orthogonal projection onto the subspace S . Repeat the previous steps to produce the projection vector $\text{temp2} = P2 * \text{temp}$ and plot this vector in the previous figure window. This will produce the linear approximation to the given data. Run the following code in Python:

You should get a number which is close to zero again, such as, $5.6169\text{e-}16$. There is a geometric explanation for this fact. The matrices $P1$, $P2$ are the projection matrices, and projecting a vector on a subspace produces a vector belonging to this subspace. Taking a second projection, thus, will simply leave the vector as is. The projection matrix behaves as an identity matrices on the corresponding subspace if the basis consisting of the columns of the matrix $Q2$ is selected.

Variables: P2, temp2

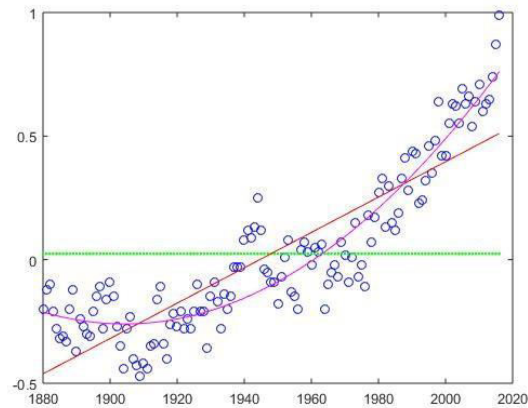


Figure 10: Approximation of data by a quadratic function.

```
# Projection matrix onto the subspace S
P2 = Q2 @ Q2.T

# Project the temperature data
temp2 = P2 @ temp

# Plot the original and projected temperatures
plt.figure()
plt.plot(years, temp, 'bo', label='Original Data')
plt.plot(years, temp1, 'g.', label='Constant Approximation')
plt.plot(years, temp2, 'r', label='Linear Approximation')
plt.xlabel('Year')
plt.ylabel('Temperature Anomaly (°C)')
plt.title('Temperature Data Projection')
plt.legend()
plt.grid(True)
plt.show()

# Display the projected temperatures
print("Projected temperatures (temp2):")
print(temp2)

# Norm of P2^2 - P2
norm_P2 = np.linalg.norm(P2 @ P2 - P2)
print(f"norm(P2 * P2 - P2) = {norm_P2:.4e}")
```

17. Finally, let us approximate the data by using a quadratic function. Create a new cell and reproduce the code from the previous cell but with renamed variables (e.g., B3 instead of B2). Make sure to copy the plotting code and switch P2 to P3.

Aside from changing 2s to 3s, the only other change you'll need to make is to add a column to the B3 matrix. This column should be the vector of squares of the elements of the vector years, which can be produced in Python with the expression `years.^2` (notice the dot before the power sign!).

Variables: B3, Q3, P3

18. Plot the initial data and all the approximations above on a new plot (use the command `figure`). If correct, your code should produce a figure similar to the one on the Fig. 10. Observe that the quadratic function (shown by the magenta line) appears to follow the global tendencies of the temperature graph pretty well.

Combined solution for 17,18

```
# Add a column of squared years to the matrix B3
B3 = np.column_stack((np.ones(m), years, years**2))

# Create the orthonormal basis Q3 using the orth function
Q3 = np.linalg.qr(B3)[0] # Using QR decomposition to get an orthonormal basis

# Projection matrix onto the quadratic subspace
P3 = Q3 @ Q3.T

# Project the temperature data onto the quadratic subspace
temp3 = P3 @ temp

# Plot the original and projected temperatures
plt.figure()
plt.plot(years, temp, 'bo', label='Original Data')
plt.plot(years, temp1, 'g', label='Constant Approximation')
plt.plot(years, temp2, 'r', label='Linear Approximation')
plt.plot(years, temp3, 'm', label='Quadratic Approximation')
plt.xlabel('Year')
plt.ylabel('Temperature Anomaly (°C)')
plt.title('Temperature Data Projection')
plt.legend()
plt.grid(True)
plt.show()

# Display the projected temperatures
```

```

print("Projected temperatures (temp3):")
print(temp3)

# Norm of P3^2 - P3
norm_P3 = np.linalg.norm(P3 @ P3 - P3)
print(f"norm(P3 * P3 - P3) = {norm_P3:.4e}")

```

19. Now, let us make predictions for the future. Create a new cell and run the following code:

This should create our prediction for an average temperature for the next 100 years. If you did everything correctly, provided the current tendencies hold, by the year 2116 the average temperature should be about 3.5C. Note that climate prediction is notoriously difficult, and it is very likely that there are factors which influence the data which are not accurately picked up by our simple model. Also, please note that in the past, temperatures tended to rise faster than any prediction. At the same time, NASA reports that the currently available models for climate change predict rise in temperatures anywhere between a 2C and 6C, so our simple model predictions are right in the middle!

```

#19

# Define future years for prediction
futureyears = np.arange(2016, 2117)

# Perform spline interpolation and extrapolation for future temperatures
interp_func = interp1d(years, temp3, kind='cubic', fill_value='extrapolate')
futuretemp3 = interp_func(futureyears)

# Create a new figure for plotting
plt.figure()

# Plot future temperature predictions
plt.plot(futureyears, futuretemp3, 'g-')
plt.xlabel('Year')
plt.ylabel('Predicted Average Temperature (°C)')
plt.title('Predicted Average Temperature for Next 100 Years')
plt.grid()

# Display the predicted temperature for 2116
print(f'Predicted average temperature for the year 2116: {futuretemp3[-1]:.2f} °C')

plt.show()

```

Project 9: Orthogonal matrices and 3D graphics

Goals: To explore applications of orthogonal matrices to rotation of 3D objects.

To get started: Download the files lab09.py⁹, v.mat, and f.mat. The last two files contain corresponding vertices and faces of a 3D model¹⁰. Save these files in your working directory.

Python commands used: def, np.zeros, np.sin, np.cos, if ... else, for ... in range, plt.plot, plt.figure, plt.hold, buckyball, len, np.load."

What you have to submit: The files rotation.m and lab09.py which you will create during the lab session.

INTRODUCTION

Recall that the square matrix $\mathbf{Q}$ is called orthogonal if $\mathbf{Q}^T \mathbf{Q} = \mathbf{Q} \mathbf{Q}^T = \mathbf{I}$, where $\mathbf{I}$ is an identity matrix of the same size as $\mathbf{Q}$. In other words, this means that $\mathbf{Q}^T = \mathbf{Q}^{-1}$. Rows (columns) of an orthogonal matrix generate an orthonormal set of vectors. Namely, if $\mathbf{Q} = [\mathbf{q}_1, \mathbf{q}_2, \dots, \mathbf{q}_n]$, where $\mathbf{q}_j$ are the columns of the matrix $\mathbf{Q}$, then $\mathbf{q}_j \cdot \mathbf{q}_k = \delta_{jk}$, and δ_{jk} denotes the Kronecker symbol: $\delta_{jk} = 1$, if $j = k$, and $\delta_{jk} = 0$, if $j \neq k$.

Another property of orthogonal matrices is that $\mathbf{x} \cdot \mathbf{y} = \mathbf{Q} \mathbf{x} \cdot \mathbf{Q} \mathbf{y}$ for any two vectors $\mathbf{x}$ and $\mathbf{y}$, meaning that multiplication by an orthogonal matrix $\mathbf{Q}$ does not change the scalar product of two vectors. Geometrically, this means that a linear transformation $T(\mathbf{x}) = \mathbf{Q} \mathbf{x}$ of the space $\mathbb{R}^n$ is equivalent to the rotation of the space together with a possible reflection along some hyperplane.

⁹ The printout of the file lab09.m can be found in appendices of this book.

¹⁰ The original data for this 3D model can be found here: <https://www.thingiverse.com/thing:906692>. The model is licensed under Creative Commons Attribution Share Alike license.